

# Revenue RCA that alerts while the data is still landing.

One ClickHouse pipeline handles ingestion, decomposition, and insert-time alerting for InMobi's ad revenue — then a LibreChat agent lets anyone ask it questions in plain English. This deck covers the business case, the cost profile, and what it found.

|                                             |                                                      |                                                   |                                                        |
|---------------------------------------------|------------------------------------------------------|---------------------------------------------------|--------------------------------------------------------|
| <div>7</div> <div>confirmed incidents</div> | <div>\$310.31</div> <div>revenue impact caught</div> | <div>21ms</div> <div>server-side drill-down</div> | <div>0</div> <div>false positives, 24 clean days</div> |
|---------------------------------------------|------------------------------------------------------|---------------------------------------------------|--------------------------------------------------------|

## 01 • THE PROBLEM

# Ad revenue anomalies are high-cardinality, high-velocity, and easy to miss

- 01 9 independent dimensions, millions of events a day.** A fault scoped to one OS on four countries doesn't move any dashboard total enough to notice.
- 02 Manual and batch RCA runs hours to days behind.** By the time a human spots the dip, the revenue is already gone.
- 03 Dimension tables get regenerated between data drops.** The same device ID can carry a different country or category next week — a naive detector silently misattributes segments and raises false incidents.

**40 days**

of ad\_events scored end to end — 10.5M events, 9 dimensions, 4 revenue factors, one identity to decompose them by.

# What the pipeline is worth

## \$310.31

### Revenue impact caught

Across 7 real incidents in a single 40-day pass — every dollar quantified with a reproducible query, not an estimate.

## Insert-time

### Detection, not discovery

Alerts score inside the same materialized view that ingests the event. No nightly batch window, no cron job standing between a fault and its alert.

## Plain English

### LibreChat as the front door

A conversational agent sits on the same tables — "why did fill rate drop Tuesday" needs no SQL and no dashboard to read.

*Every incident on this list was previously invisible until someone noticed a dashboard dip — usually a day or more after the revenue was already lost.*

# The ClickHouse-native design pays for itself in infrastructure, not just detection



**~2%**

**Storage overhead for the whole drill-down layer**

2.90 MiB of rollups against a 144.19 MiB raw fact table — every dimension, every hour, queryable without touching the source rows.



**79×**

**Fewer rows scanned per drill-down**

A 9-dimension, 4-day scan reads 15,360 rollup rows instead of 1,209,372 raw events for the same window.



**21ms**

**Server-side time for a full dimension scan**

Interactive, not batch — the same query a LibreChat agent calls mid-conversation.



**1 engine**

**No separate streaming stack**

Dictionaries, materialized views, and parameterized views do ingestion, decomposition, and alerting inside ClickHouse — no Kafka, Flink, or Airflow layer to run or pay for.

# One engine, four jobs



## Dictionaries — `COMPLEX_KEY_HASHED`

O(1) segment lookups inside every materialized view. No per-query JOIN plan against a dimension table sized in the thousands of rows.



## Materialized views that fire on INSERT

An alert can exist in the same transaction as the event that caused it — the alerting layer has no separate schedule to fall behind.



## SummingMergeTree + ARRAY JOIN — marginal, not cubed

One row per (dimension, value, hour), built from a literal array of 9 dimension tuples: 61,440 rows, versus the 7.9M-row full cube the naive approach produces for the same facts.



## Parameterized views — `rca_scan` / `rca_seg` / `rca_volume`

The drill-down protocol as a callable query with named arguments, not a one-off script re-written for every incident.



## Cross-database, read-only baselines

A fresh dataset is scored against a trained baseline in another database without ever writing to the source — the June data stays untouched.

# A conversational analyst on top of the same tables

The agent's entire context is one file: a system prompt that teaches it the table catalog, the four-step drill-down protocol, and the traps that produce a confident wrong answer if skipped.

Every SQL example inside it was **executed against the live cluster** before it shipped — the agent is never reasoning from a description of the schema, it's reasoning from queries that were proven to run.

313 lines

~3,456 tokens

librechat-rca-agent.md

ANALYST

Why did fill rate drop on July 8th?

RCA AGENT

os\_version = iOS 17.5 collapsed from 79.2% to 47.8% fill (z = -117), uniform across all 16 countries and all 3 iPhone models carrying that OS — an OS-level fault, not a geography or device issue.

ANALYST

Is that the real cause, or bleed-through?

RCA AGENT

Real cause. device\_model=iPhone 14 and country=ID also flag, but at half the |z| — they're just the buckets with the most iOS 17.5 traffic.

# 7 incidents, 40 days, zero false positives

| DAYS      | METRIC    | ROOT CAUSE                    | SEV | IMPACT   |
|-----------|-----------|-------------------------------|-----|----------|
| Jun 16–18 | eCPM      | region=EU × interstitial      | P2  | −\$20.00 |
| Jun 19–22 | eCPM      | category=finance              | P1  | −\$41.80 |
| Jun 21    | requests  | ingestion gap, whole pipeline | P1  | −43.9%   |
| Jun 23–25 | fill_rate | os_version=Android 15         | P1  | −\$68.11 |
| Jun 28–30 | fill_rate | iOS 18.1 × APAC               | P1  | −\$20.66 |
| Jul 8–9   | fill_rate | os_version=iOS 17.5           | P1  | −\$66.00 |
| Jul 9–10  | eCPM      | ad_format=video               | P1  | −\$93.74 |

24 / 40 days clean

277 raw segment/day alerts collapse to these 18 incident rows — the rest is arithmetic bleed-through from the same root cause, not separate faults.

SUMMARY

# Detection that pays for itself

|                                                                 |                                                       |                                                  |                                                          |                                                        |
|-----------------------------------------------------------------|-------------------------------------------------------|--------------------------------------------------|----------------------------------------------------------|--------------------------------------------------------|
| <div>7</div> <div>incidents found, 2 sealed + 5 recovered</div> | <div>\$310</div> <div>revenue impact quantified</div> | <div>21ms</div> <div>drill-down query time</div> | <div>~2%</div> <div>storage overhead vs raw events</div> | <div>0</div> <div>false positives, 24 clean days</div> |
|-----------------------------------------------------------------|-------------------------------------------------------|--------------------------------------------------|----------------------------------------------------------|--------------------------------------------------------|

*Built entirely on ClickHouse — ingestion, decomposition, insert-time alerting, and a natural-language interface, in one system, with every number reproducible from the SQL in this repository.*